

Patrol Field Effort

Technical Guide

Patrol Field Effort Analysis — Methodology & Calculation Reference
Version 1.0 · Generated September 10, 2026

1. Overview

The **Patrol Field Effort** workflow analyses ranger patrol activity within a user-selected study area, sourced live from **EarthRanger**. It converts completed patrols into trajectories, overlays them onto an analysis grid, and computes three complementary spatial field-effort metrics per grid cell: days since last visit, dwell time, and linear time density.

The workflow produces three interactive maps, three scalar summary widgets, and one sortable operational-days table, all assembled into a single dashboard. There is no per-group breakdown — groupers are fixed to an empty list, so every output covers the whole dataset as one combined view.

Note: All three field-effort metrics are computed over the **same** analysis grid, so the three maps are directly comparable cell-for-cell.

2. Dependencies & Prerequisites

2.1 EarthRanger Connection

Patrol data is fetched from an **EarthRanger** instance via `set_er_connection`. `get_patrol_observations` queries completed patrols only (`status: ["done"]`), with `include_patrol_details: true`, `patrols_overlap_daterange: true`, a sub-page size of 150, and `raise_on_empty: true` — the run fails rather than producing an empty dashboard if no patrols match the configured time range.

2.2 Groupers

`set_groupers` is called with `partial: groupers: []`, fixing the grouping strategy to an empty list. Unlike sibling patrol workflows that expose a user-selectable grouper field, this workflow always computes a single combined view — there is no per-group map or table variant.

2.3 Area of Interest

The study area boundary is fetched live from EarthRanger via `get_spatial_features` (no bundled or Dropbox-downloaded boundary file). The selected geometry is validated as polygonal (`assert_polygon_types`) and reprojected to a metric CRS (EPSG:3857) before being used as both the map extent and the grid clip boundary. It is persisted as `er_spatial_file.gpkg`.

2.4 Analysis Grid & Cell Size

The grid cell size field normally lets a user choose between an auto-scaled grid and a custom cell size. This workflow overrides the RJSF schema (`rjsf-overrides` in `spec.yaml`) to only accept the **custom** cell-size schema, hides the auto/custom toggle in the form, and defaults the cell size to **1000 m**. The grid is built with `create_meshgrid` (`intersecting_only: true`) and then clipped to the study area with an intersection overlay (`keep_geom_type: true`, `make_valid: true`).

2.5 Base Map Tile Layers

Background tile layer(s) for all three maps are configured via `set_base_maps_pydeck` and are fully user-editable (URL, opacity, max zoom) — there are no workflow-level defaults hardcoded in `spec.yaml` beyond the task's own schema defaults.

3. Data Ingestion Pipeline

3.1 Patrol Observations → Trajectories

relocations_to_trajectory converts the fetched patrol observations into trajectory segments, adding segment_start, segment_end, and dist_meters. Trajectories are reprojected to EPSG:3857 (reproject_trajs) for all metric calculations, then re-reprojected to EPSG:4326 (WGS84) wherever a result is rendered on an interactive map. Raw observations and trajectories are persisted as patrol_observations.gpkg and patrol_trajectories.gpkg.

3.2 Column Standardisation & Patrol Date

map_columns renames raw EarthRanger extra__* fields to plain names used throughout the workflow:

Original column	Renamed to
extra__patrol_type__value	patrol_type
extra__patrol_serial_number	patrol_serial_number
extra__patrol_status	patrol_status
extra__patrol_subject	patrol_subject
extra__subject_id	subject_id
extra__patrol_id	patrol_id
extra__patrol_title	name

decompose_datetime then extracts the calendar date component from segment_start (as segment_start_date), used later to count distinct patrol days per ranger.

3.3 Grid Overlay

Trajectories are reduced to segment_start, segment_end, and geometry, then spatially joined against the clipped analysis grid with spatial_join (how: inner, predicate: intersects). Every downstream metric is computed from this joined table, grouped back to the grid's index column.

4. Field Effort Metrics — Methodology

4.1 Days Since Last Visit

summarize_df aggregates the joined trajectory/grid table by index, taking max(segment_end) as last_visited and min(segment_start) as first_visited per cell. add_time_since_visit then computes days_since_visit as the elapsed time between last_visited and the end of the configured time range. Cells with no patrol coverage have a null value, flagged via add_non_null_flag (column visited).

4.2 Dwell Time (Hours)

compute_dwell_time takes the reprojected patrol trajectories and the gridded study area and computes hours_in_cell — total time rangers spent with a fix inside each grid cell. Cells with zero recorded dwell time are again flagged via add_non_null_flag.

4.3 Linear Time Density

calculate_linear_time_density computes route-coverage density as a percentile value per cell that intersects a patrol trajectory (percentiles: null — uses the task's default percentile scheme). Cells with **no** trajectory coverage are not

returned by this task, so they are recovered separately: a difference overlay between the full clipped grid and the covered cells yields the uncovered cells, which are then concatenated back onto the result (concat_dataframes, ensuring the percentile, density, and area_sqkm columns exist on both sides) so every grid cell appears in the final output.

5. Classification & Styling

All three metrics (days_since_visit, hours_in_cell, percentile) go through the identical classification pipeline:

Step	Task	Configuration
Bin	add_visit_bins	5 classes, natural-breaks (Jenks) scheme, absolute value
No-data label	add_visit_bins	"Unvisited" for cells not flagged as visited
Colour	add_bin_colors	RdYlGn palette across the 5 bins
No-data colour	add_bin_colors	#808080 (grey) for "Unvisited" cells

Each metric's hex colours are decategorised to strings and converted to RGBA (add_rgba_from_hex) before being used as a deck.gl fill colour. Bin legend order is fixed with order_bin_categories so the legend always reads from the lowest to the highest bin, ending in "Unvisited".

6. Map Outputs — Methodology

All three maps share the same GeoJSON polygon layer style: filled, stroked in black at 0.25 px width, 55% fill opacity, fill colour driven by each cell's RGBA colour. A single unfilled study-area boundary layer (0% fill opacity, full line opacity) is composited underneath every map for spatial context. Map zoom and centre are computed once from the study area's extent and reused across all three maps (max zoom 15 for extent-fitting, 10 for the rendered map).

Map	Value column	Legend title
Days Since Patrol Visit	days_since_visit	Time Since Visit (Days)
Time Spent Per Grid Cell	hours_in_cell	Time Spent (Hours)
Patrol Linear Time Density	percentile	Time Spent

Each map is rendered with draw_map (legend placement: bottom-right) and persisted as an interactive HTML file (days_since_patrol_visit_map.html, time_spent_per_grid_map.html, and ltd_patrols_map.html respectively).

7. Summary Metrics

7.1 Scalar Dashboard Widgets

Widget title	Source	Aggregator / Unit
Total Patrols	subject_id	nunique · count
Total Patrol Distance	dist_meters	sum · converted m → km
Patrol Coverage	compute_patrol_effort_fraction	fraction of grid with ≥1 visit · %

7.2 Operational Days Table

summarize_df groups the renamed, date-decomposed trajectory table by subject_id and patrol_subject:

Output column	Source column	Aggregator	Unit
distance_km	dist_meters	sum	km (1 dp)
duration_hrs	timespan_seconds	sum	h (1 dp)
patrol_days	segment_start_date	nunique	count

Column headers are relabelled for display (Subject ID, Patrol Subject, Distance (km), Duration (hrs), Patrol Days) and rendered as a sortable, filterable HTML table (draw_table), persisted with the patrol_summary_table filename suffix.

8. Interactive Dashboard

gather_dashboard assembles the final dashboard from seven widgets, in order:

#	Widget	Type
1	Total Patrols	Single value
2	Total Patrol Distance	Single value
3	Patrol Coverage	Single value
4	Days Since Patrol Visit	Map
5	Time Spent Per Grid Cell (Hours)	Map
6	Patrol Linear Time Density	Map
7	Patrol Summary	Table

9. Output Files

All files are written to \$ECOSCOPE_WORKFLOWS_RESULTS.

File	Format	Content
er_spatial_file.gpkg	GeoPackage	Study area boundary (EPSG:3857)
patrol_observations.gpkg	GeoPackage	Raw patrol observation points
patrol_trajectories.gpkg	GeoPackage	Patrol trajectory lines
days_since_patrol_visit.gpkg	GeoPackage	Grid with days-since-visit stats
days_since_patrol_visit_map.html	HTML	Interactive days-since-visit map
time_spent_per_cell.gpkg	GeoPackage	Grid with dwell time (hours) per cell
time_spent_per_grid_map.html	HTML	Interactive dwell time map
patrols_linear_time_density.gpkg	GeoPackage	Grid with linear time density percentiles
ltd_patrols_map.html	HTML	Interactive linear time density map

File	Format	Content
patrol_operational_days.csv	CSV	Operational days per ranger

10. Workflow Execution Logic

10.1 Skip Conditions

Two default skip conditions apply to every task (task-instance-defaults):

- **any_is_empty_df** — skips a task (and its dependants) when any input DataFrame is empty.
- **any_dependency_skipped** — propagates skips downstream automatically.

Unlike some sibling workflows, patrol widget tasks here do **not** override this with skipif: [never] — combined with raise_on_empty: true on the patrol fetch, an empty result set surfaces as a hard failure rather than a partially-empty dashboard.

10.2 Data Flow Summary

Stage	Tasks
Setup	ER connection, time range, groupers (fixed empty), base maps, area of interest
Ingest	Fetch completed patrol observations → build trajectories
Study area	Validate polygon → reproject to EPSG:3857 → persist
Grid	Create meshgrid (custom cell size) → clip to study area → reset index
Join	Reproject trajectories → select columns → spatial join onto grid
Days since visit	Summarise last/first visit → time-since-visit → merge → flag → bin → colour → map
Dwell time	Compute dwell time → merge → flag → bin → colour → map
Linear time density	Calculate LTD → recover unvisited cells (difference + concat) → flag → bin → colour → map
Summary	Rename columns → extract date → operational days table → scalar widgets
Dashboard	gather_dashboard combines all 7 widgets

11. Software Versions

Package	Version	Role
ecoscope-platform	>=2.18.0	Consolidated core task library and workflow engine
ecoscope-workflows-ext-custom	0.1.0rc14.*	Utility tasks (persistence, maps, layers)
ecoscope-workflows-ext-ste	0.0.0rc1.*	Spatial operations tasks (view state, layer combination)
ecoscope-workflows-ext-mep	1.0.3.*	Domain tasks (visit bins, dwell time, colour bins, patrol effort fraction)

Package	Version	Role
pydeck	0.9.2	Deck.gl map rendering
opentelemetry-sdk	>=1.20.0, <2.0.0	Observability/tracing

Packages are distributed via the `repo.prefix.dev` conda channels (`ecoscope-workflows` and `ecoscope-workflows-custom`) and pinned to compatible version ranges in `spec.yaml`. The runtime environment is managed by **pixi**.